

Memory bar in side panel shows what AI knows; link to memory editor for full control per §11.3.

Section 25 — Data Privacy & Retention

US-only launch; CCPA-aligned. Architecture supports GDPR additions later.

25.1 Data Classifications

Class	Examples	Treatment
PII — Identity	Name, email, phone, DOB, address	Encrypted at rest by Supabase; access logged
PII — Financial	Payment methods (token only), bank info (Stripe-held)	Platform never stores card numbers; Stripe-held
PII — Sensitive	Accessibility needs, dietary restrictions	Encrypted; access tightly scoped
Conversation data	Chat messages, memory	Tenant-scoped; customer can view, edit, delete
Behavioral	Click events, page views, response feedback	Aggregated for product analytics; per-user trace retained 90 days

25.2 Retention Schedule

Data	Retention	Reason
Booking records	7 years post-sailing	Tax, liability, host agency obligations
Commission records	7 years	Financial; matches IRS retention
Audit logs	7 years	Regulatory and disputes
Conversation messages (active customer)	Indefinite while tenant active	Customer service continuity; memory
Conversation messages (terminated customer)	Bodies deleted after deletion grace period; metadata retained per §25.4a	Per CCPA
Anonymous session data	60 days	Cleanup of unattached data
RAG queue rejected items	90 days	Pattern analysis
Retrieval logs (detailed)	90 days, then aggregated	Performance and feedback signal

Marketing engagement events	13 months	Year-over-year comparison
-----------------------------	-----------	---------------------------

25.3 CCPA Rights

Right	Mechanism	SLA
Right to know	GET /api/user/data → emailed download link	Within 45 days; usually <24h
Right to delete	DELETE /api/user/data → 30-day grace → purge	30 days
Right to correct	Self-edit in /settings/profile and /settings/memory	Immediate
Right to opt out of sale	N/A; platform does not sell data	N/A
Right to non-discrimination	No service degradation for exercising rights	Always

25.4 Anonymization of Required-Retention Data

Booking and commission records are subject to 7-year retention; they survive customer deletion via anonymization:

- Customer name replaced with hash-derived placeholder
- Customer email cleared; reference to deleted user only
- Customer DOB cleared (no longer needed post-sailing for retention purpose)
- Booking financial and audit details preserved

25.4a Free-Text Anonymization Strategy

Free-text fields (chat messages, conversation notes, AI-generated narratives) cannot be reliably anonymized in place — PII redaction on years-old free-text via LLM is expensive, slow, and produces false negatives. The platform takes a different approach for the two categories of free-text data:

Category 1: Conversation messages (messages.content)

On customer deletion after the 30-day grace period:

- **messages.content set to NULL.** The free-text body is removed entirely.
- **Message metadata retained.** id, conversation_id, sender_role, created_at, token counts, supervisor_findings JSONB (with any PII fields within it nulled separately) all stay.
- **Conversation continuity preserved for the tenant.** The tenant

can still see “conversation existed with N messages of which K were customer-sent” for their own historical analytics, but cannot read the customer’s words.

This is deletion, not anonymization. The body is gone.

If a tenant needs to preserve conversation content for legal-hold purposes (active dispute), the forensics log per §26.5a captures a snapshot BEFORE the deletion. The snapshot is encrypted and access-controlled per that section; it is NOT in the tenant’s normal operational view.

Category 2: AI-generated narratives in other tables

Some tables contain AI-generated free-text:

- `quotes.narrative` — generated quote description
- `bookings.notes` — generated booking summary
- `customer_memories.notes_freeform` — extracted free-form notes

On customer deletion:

- `customer_memories` row is hard-deleted entirely (the row exists to track customer state; if there’s no customer, there’s no row).
- `quotes.narrative` and `bookings.notes`: NULLED for the affected customer’s records. The booking record itself is retained per §25.4 (financial retention) but the narrative is gone.

Category 3: Tenant-owned content referencing the customer

Tenant-authored notes (CRM notes typed by a tenant agent about a customer) are tenant-owned. On customer deletion:

- The notes are NOT auto-deleted. The tenant agent wrote them; the tenant retains them as part of their business records.
- The notes’ references to the customer (foreign keys) are replaced with the same hash-derived placeholder from §25.4.
- The text body of the notes is retained but the tenant is encouraged (via UI) to review and redact any PII manually.

Why deletion (not redaction) for `messages.content`

Three reasons:

1. **Reliable PII redaction on years-old free-text is hard.** Names, addresses, phone numbers, casual references — an LLM pass produces false negatives. Deletion is reliable.
2. **The customer’s intent on deletion is clear.** They want their data gone. Retaining redacted versions of their messages doesn’t serve them.
3. **Cost.** Running an LLM redaction pass over the message history of every CCPA-deleted customer is real Anthropic spend.

The trade-off: the tenant loses the ability to read historical conversations with terminated customers. This is the right default for CCPA compliance.

Calls Worth Flagging

- **Forensics-log snapshot before deletion is the recovery path for active disputes.** A customer who is mid-dispute deletes their account, then resumes the dispute later: the snapshot lets the platform reconstruct what was said. The snapshot is gated by active-dispute status; it is not taken on every deletion.
- **Tenant-owned notes are deliberately handled differently.** CCPA gives the customer rights to their own data, not to data the tenant authored about them in a business-records capacity.
OPERATOR CONFIRM: This stance should be reviewed with legal counsel before launch.
- **The §17.10 deletion grace period (30 days)** currently doesn't distinguish "soft delete that can be reversed" from "deletion in progress." During the 30-day grace, messages.content is still readable; only after grace does the NULL-out happen.

25.5 Sub-Processors

Disclosed in privacy policy; updated annually:

- Anthropic (AI inference)
- OpenAI (embeddings only; no chat content)
- Supabase (database, auth, storage)
- Vercel (hosting, edge functions)
- Stripe (payments, Connect, 1099 reporting)
- Resend (email delivery)
- Inngest (background job processing)
- Sentry (error tracking, PII-scrubbed)

25.6 Data Residency

US-only at launch. All data centers in US. Architecture supports EU residency for GDPR additions later (separate Supabase regions, regional routing in main app).

25.7 Consent Granularity

Distinct consents per data use:

- Account creation: required (per ToU)
- AI conversation processing: required (per AI Disclaimer)
- Memory extraction: opt-out (default on; users.memory_opt_out flag disables)
- Marketing email: opt-in (default off; users.marketing_email_opt_in)
- Travel news email: opt-in (default off; users.travel_news_opt_in)
- Performance analytics: opt-out (default on)

25.8 Cookie Policy

- Essential: session, security, basic preferences (always on)
- Performance: analytics (opt-out)
- Marketing: cross-site retargeting (opt-in; off by default)
- Cookie consent banner on first visit; preferences persisted

25.9 Data Breach Response

Per §26.10. Notification SLAs:

- Affected users: within 72 hours of confirmed breach
- Tenant admins: within 24 hours of confirmed breach affecting their tenant
- California AG: per state law requirements
- Public disclosure: when applicable per state law

25.10 Production Data on Staging — Privacy Posture

The staging environment receives a copy of the production database at the start of every release pipeline. PII is not anonymized in the staging copy. This is a deliberate trade-off documented in §30.9.1; it sacrifices a portion of the standard “test data is synthetic only” posture in exchange for E2E test fidelity against real data shapes and volumes.

Customer-facing implications:

- Staging data is subject to the same retention schedule as production for the purpose of customer deletion requests. A CCPA deletion request must purge the customer from staging on the next release cycle (the prod copy that occurs after deletion will no longer contain the record).
- Staging is included in breach response scope per §26.10. A breach affecting the staging database is reportable to the same extent as a breach of production.
- Sub-processor disclosure (§25.5) accurately reflects staging data exposure: the same vendors hold or process staging data as hold or process production.

The controls preventing customer impact from staging activity are listed in §30.9.1 and §26.13.

Section 26 — Security

Defense-in-depth across authentication, authorization, encryption, audit, monitoring, and incident response.

26.1 Threat Model

Category	Examples	Primary Defenses
External attacker	Credential stuffing, SQL injection, XSS, CSRF	Supabase Auth + parameterized queries + CSP + same-site cookies
Insider threat	Malicious tenant trying to exfiltrate competitor data	Tenant-scoped RLS; permission checks; audit logs
Cross-tenant leak	Tenant A's data leaks to Tenant B	RLS on every table; permission-check at handler
AI-specific	Prompt injection, jailbreak, data leak via AI	System-prompt isolation; supervisor checks; output filtering
Supply chain	Compromised dependency, malicious npm package	Lockfile, audit, dependency scanning

26.2 Four-Layer Auth

The platform's defense in depth for tenant isolation runs through four layers. Each is necessary; none is sufficient on its own.

Layer 1: Identity (who are you?)
Supabase Auth + OAuth providers
|
▼
Layer 2: Tenant context (which tenant?)
Middleware resolves tenant from host header
Validates user has a public.users row in this tenant
Produces a TenantContext (§5.4.2) with source = http_request
|
▼
Layer 3: Authorization (what can you do?)
assertPermission() at handler entry
Returns enriched ctx with role + resource access decisions
|
▼
Layer 4: Data access scoping (which rows can you touch?)
tenantClient(ctx) wraps the DB client (§5.4.3)
All queries against tenant-scoped tables auto-filtered by tenant_id
RLS policies (§5.1) backstop the application-layer scoping

Each layer's purpose:

- **Layer 1** answers “is this request from a real authenticated user?” — handled by Supabase Auth and the OAuth providers.
- **Layer 2** answers “which tenant is this request operating inside?” — handled by middleware that resolves the tenant from the request's host header (subdomain or custom domain per §1.4) and validates the user has membership.
- **Layer 3** answers “is this user allowed to do this action on this resource?” — handled by assertPermission() at handler entry.

- **Layer 4** answers “for queries this handler issues, can the database actually return cross-tenant data?” — handled by the `tenantClient` wrapper AND by RLS policies. Two independent defenses; either alone is insufficient.

Layers 1–3 are application-layer checks. Layer 4 is the boundary between application and database. Layers 2 and 3 produce the `TenantContext` that Layer 4 consumes — the type system threads context through the call chain so it cannot be lost.

For non-HTTP contexts (Inngest jobs, webhook handlers), Layers 1 and 2 are replaced by the appropriate `TenantContext` factory per §5.4.5. Layer 4 is identical regardless of context source — the same `tenantClient(ctx)` wrapper applies.

26.3 Authorization Pattern

Every protected HTTP handler entry calls `assertPermission()` to produce a `TenantContext` (§5.4.2) plus role/resource access decisions. The handler then uses the returned `ctx` to construct a scoped database client via `tenantClient(ctx)` (§5.4.3).

```
// Every protected handler entry:
export async function POST(req: Request) {
  const { ctx, user, permissions } = await assertPermission(req, {
    resource: 'booking',
    action: 'modify',
    resourceId: bookingId,
  });
  const db = tenantClient(ctx);
  // ...handler body, using db for all queries...
}
```

Inside `assertPermission()`

1. Decode JWT → `auth.users.id`.
2. Resolve tenant from request host per §1.4 → `tenant_id`.
3. Verify the user has a `public.users` row for this tenant with `status = 'active'`. If not, throw 403.
4. For write actions, verify `tenant_is_active(tenant_id)` — see §5.1’s policy convention. A user can read their data from a suspended tenant; they cannot write. If write requested against a non-active tenant, throw 403 with a structured error (`tenant_suspended`, `tenant_terminated`, etc.) so the client UI can render the right message.
5. Check role permissions per the role matrix and resource-specific rules.
6. Construct a `TenantContext` via `tenantContextFromRequest()` (§5.4.5). The context records `source = http_request, user_id`.
7. Return `{ ctx, user, permissions }` or throw 403.

Returned context shape

```
type AssertPermissionResult = {
  ctx: TenantContext;           // For tenantClient(ctx)
  user: { id: string; email: string; roles: string[] };
  permissions: { allowed: true; expires_at: Date }; // Time-bounded
  // for sensitive ops per §17.7
};
```

Sensitive-action re-auth

Per §17.7, certain operations (commission overrides, role changes, ICA acceptance) require re-auth within 4 hours of the action. The `assertPermission()` function checks the JWT's `auth_time` claim against the action's sensitivity classification. Failing the freshness check throws a structured 401 with a re-auth challenge.

26.3a Service-Role Discipline

The Supabase service role bypasses RLS. It is the most powerful credential in the platform. Its use is restricted by both convention and lint enforcement.

26.3a.1 The four authorized service-role contexts

Service-role usage is permitted only in these four contexts. Code in any other location that imports `createServiceRoleClient` is a CI failure.

Context	Wrapper	Tenant scoping
<code>tenantClient(ctx)</code> per §5.4.3	Yes — automatic <code>.eq('tenant_id', ...)</code> injection	Tenant-scoped
<code>platformAdminClient()</code> per §5.4.1	No wrapper; raw access	None — cross-tenant by design
Database migrations (supabase db push)	Run via dedicated migration role per CI/CD §5	N/A — DDL only
Backup/restore tooling	Run from operator workstation; not from application code	N/A — out-of-band

26.3a.2 The forbidden patterns

Each of these is a hard CI failure:

- `createServiceRoleClient()` called outside `src/lib/db/tenant-client.ts` or `src/lib/db/platform-admin-client.ts`.
- Importing `SUPABASE_SERVICE_ROLE_KEY` directly into application code (the `env` var is read once, in the client factories, never re-read elsewhere).
- A `tenantClient` constructed from anything other than one of the four `TenantContext` factories in §5.4.5.
- A function signature accepting `tenant_id: string` and constructing database queries from it — instead it must accept `ctx: TenantContext` and use `tenantClient(ctx)`.

26.3a.3 The exception: `platformAdminClient`

`platformAdminClient` exists because some operations legitimately cross tenant boundaries — the global RAG promotion queue, platform-wide abuse-monitoring rollups, tenant-listing for platform admin dashboards. These call sites:

- MUST be named with `platformAdmin*` prefixes so they're searchable.

- MUST be wrapped in `assertPlatformAdmin(req)` (the platform-admin variant of `assertPermission`) before the client is constructed.
- MUST audit at the JOB level (one `audit_log` row per function invocation), not per query.

The set of `platformAdminClient` call sites is small (target: under 20 across the whole codebase) and reviewed quarterly per §26.11.

Per-job audit pattern

A platform-admin operation is one function from entry to exit. One audit row is written for each such operation, capturing what the operation did across all its queries. The row is constructed at the function's entry point with a reason and a `correlation_id`, and is committed at the function's exit with the accumulated query summary filled in.

The pattern is enforced by wrapping `platformAdminClient()` calls in a single utility: `withPlatformAdminAudit(reason, fn)`. The utility manages the audit row's lifecycle so handler code cannot construct a `platformAdminClient` without going through it. See §5.4.8 for the utility definition.

```
// Platform-admin handler – the only correct shape
export async function GET(req: Request) {
  const admin = await assertPlatformAdmin(req);

  const result = await withPlatformAdminAudit(
    {
      admin_user_id: admin.user_id,
      reason: 'tenant_listing_for_admin_dashboard',
      operation: 'platformAdminListTenants',
    },
    async (db, recordQuery) => {
      // db is platformAdminClient
      // recordQuery is a callback to log a query summary into the
      audit row

      const tenants = await db.from('tenants').select('id,
display_name, status');
      recordQuery({
        op: 'select',
        table: 'tenants',
        columns: ['id', 'display_name', 'status'],
        row_count: tenants.data?.length ?? 0,
      });

      return tenants.data;
    },
  );

  return Response.json(result);
}
```

What the audit row looks like

The `audit_log` row written by `withPlatformAdminAudit`:

```
{
  "id": "uuid-of-the-audit-row",
  "tenant_id": null,           // platform-admin operations are
```

```

not tenant-scoped
  "actor_user_id": "uuid-of-admin",
  "actor_type": "admin",
  "action": "platformAdmin.tenant_listing_for_admin_dashboard",
  "resource_type": "platform_admin_operation",
  "resource_id": null,
  "changes": {
    "correlation_id": "01H...ULID", // accumulated query summary
    // tracks all queries in this
  }
job
  "operation": "platformAdminListTenants",
  "reason": "tenant_listing_for_admin_dashboard",
  "reason_detail": null, // populated only when reason =
'manual_emergency_intervention'
  "outcome": "success", // or "error_thrown"
  "duration_ms": 47,
  "queries": [
    {
      "op": "select",
      "table": "tenants",
      "columns": ["id", "display_name", "status"],
      "row_count": 12
    }
  ],
  "error_message": null // populated on failure
},
"context": {
  "ip": "10.0.0.5",
  "user_agent": "...",
  "request_id": "..." // links to access logs
},
"occurred_at": "2026-05-17T14:23:11.000Z"
}

```

Failure path

If the wrapped function throws, the audit row is STILL written, with outcome = "error_thrown", the error message captured, and any queries that had completed before the throw recorded. This means a failed platform-admin operation is auditable — partial work is documented rather than invisible.

The audit row write is non-transactional with respect to the wrapped function's queries. The audit row commits even if the wrapped function's database work was rolled back. This is deliberate: we want the audit record of an attempted action regardless of whether the action succeeded.

What “one job” means

One withPlatformAdminAudit invocation = one job = one audit row.
Concretely:

- One HTTP request that hits a platformAdmin* handler → one job.
- One Inngest function execution that uses platformAdminClient → one job per execution (Inngest retries produce separate jobs, each with its own audit row).
- A nested call where one platformAdmin* function calls another: the OUTER withPlatformAdminAudit is the job; the inner call uses the same client and reports queries into the same audit row. Nesting is detected by the utility and short-circuits to a no-op so we don't produce two rows for one logical job.

Forensic queries against this audit shape

The trade-off vs per-query-batch audit: investigations now need to read JSONB rather than filter rows by timestamp. The queries that matter remain answerable, but with a different query shape:

```
-- "What did admin X do across the last 30 days?"
SELECT
    occurred_at,
    action,
    changes->>'reason' AS reason,
    changes->>'outcome' AS outcome,
    jsonb_array_length(changes->'queries') AS query_count
FROM public.audit_log
WHERE actor_user_id = '<admin_uuid>'
    AND actor_type = 'admin'
    AND occurred_at > NOW() - INTERVAL '30 days'
ORDER BY occurred_at DESC;

-- "Did any platform-admin operation touch the `bookings` table?"
SELECT id, occurred_at, actor_user_id, changes->>'reason' AS reason
FROM public.audit_log
WHERE actor_type = 'admin'
    AND changes @> '{"queries": [{"table": "bookings"}]}':::jsonb
ORDER BY occurred_at DESC;
```

The second query uses a JSONB containment check; it benefits from a partial GIN index on `audit_log.changes` scoped to admin rows only:

```
CREATE INDEX audit_log_changes_admin_gin_idx ON public.audit_log
USING GIN (changes jsonb_path_ops)
WHERE actor_type = 'admin';
```

The index is partial — it only covers rows where `actor_type = 'admin'`. This is the only `actor_type` whose changes JSONB needs JSONB-shape querying; user-action audit rows (`actor_type = 'user'`) are queried by `actor_user_id` and timestamp, which are covered by existing indexes (per §26.5).

GIN indexes impose write cost. A partial index limits that cost to the low-volume admin-row write path. User-action writes (which dominate audit-log throughput) pay nothing.

If a future need arises to JSONB-query changes for non-admin rows (e.g., an investigation into a specific booking lifecycle through its `changes->'before'` and `changes->'after'`), add a second partial index scoped to that `actor_type` rather than promoting this one to a full index. The user-action path's write performance is a hot path and small percentages matter.

26.3a.4 Inngest jobs and webhook handlers

Inngest jobs and webhook handlers run without a user JWT, so they cannot use `userClient`. They **MUST** use `tenantClient` with a `TenantContext` constructed via the appropriate factory:

- Inngest jobs: `tenantContextFromInngestEvent(event)`. The event's payload **MUST** contain a `tenant_id`. Jobs that genuinely cross tenants (e.g., the daily promo-state-transition cron) iterate tenants explicitly and construct a fresh `TenantContext` per tenant, rather than using `platformAdminClient` to update across tenants in one query.

- Webhooks: `tenantContextFromStripeEvent(event)` for Stripe, similar factories for Resend and Gmail. Each factory validates that the webhook payload corresponds to a known tenant before constructing the context.

26.3a.5 Calls Worth Flagging

- **The discipline only works if the lint rule is enforced.** A warning that engineers can override is not a discipline; it’s a suggestion. The CI gate must be hard (red = no merge), with override requiring a documented exception in MEMORY.md.
- **Inngest events must carry `tenant_id`.** This is a design constraint on every Inngest event definition. Event types without `tenant_id` are platform-admin events and use `platformAdminClient` explicitly.
- **`platformAdminClient` call sites are the highest-value review target.** Anything that crosses tenants is anything that can leak data. Quarterly review of the set per §26.11 isn’t paperwork — it’s the audit point.
- **Per-job audit trades granularity for cost.** The pattern in §26.3a.3 writes one audit row per `platformAdmin*` function invocation, not per query. This reduces audit row volume materially (typically 5–20x for functions that issue several queries) at the cost of needing JSONB introspection to answer “which query touched table X?” questions. The partial GIN index on `audit_log.changes` and the query patterns in §26.3a.3 make this tractable. If forensic load on the audit log becomes onerous in practice (e.g., investigations routinely take many minutes), revisit per-query audit for write operations only — see §26.11 quarterly review.
- **The audit-row write is outside the wrapped function’s transaction.** This is deliberate. An admin operation that did partial work before failing still produces an audit record of what was attempted. The alternative (audit row participates in transaction, rolled back on failure) would leave failed admin operations invisible in the audit log, which is the wrong default for forensics. Engineering should not “fix” this by moving the audit row into the function’s transaction.

26.4 Encryption

Data	At Rest	In Transit
Supabase database	AES-256 (Supabase managed)	TLS 1.2+ from app
Tenant host credentials	App-layer AES-256-GCM (per-record IV) on top of DB encryption	TLS to host APIs
Tenant API keys (Resend etc.)	App-layer encryption	N/A (stored only)
Customer chat messages	Supabase managed	TLS to clients
File uploads	Supabase Storage encryption	TLS
Backups	Supabase managed	Internal to Supabase

26.5 Audit Logging

```
CREATE TABLE public.audit_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID REFERENCES public.tenants(id),  
  actor_user_id UUID REFERENCES public.users(id),  
  actor_type TEXT CHECK (actor_type IN ('user','system','admin','ai')),  
  action TEXT NOT NULL, - e.g., 'booking.submit'  
  resource_type TEXT NOT NULL,  
  resource_id UUID,  
  changes JSONB, - field-level before/after  
  context JSONB, - ip, user_agent, etc.  
  occurred_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX audit_log_tenant_idx ON public.audit_log(tenant_id,  
  occurred_at DESC);  
  
CREATE INDEX audit_log_actor_idx ON  
  public.audit_log(actor_user_id, occurred_at DESC);  
  
CREATE INDEX audit_log_resource_idx ON  
  public.audit_log(resource_type, resource_id);
```

Significant actions log: auth events, role changes, booking lifecycle, commission overrides, RAG admin actions, tenant lifecycle, payouts, subscription changes (tier, seat count, billing period). Excludes PII from log content (IDs only).

26.5a Forensics Logging

The audit log per §26.5 is the compliance/retention record — IDs only, 7-year retention. It answers “did event X happen to resource Y at time Z by actor W?” but cannot answer “what did the customer actually see?” or “what state was this booking in when the disputed action happened?”

For investigations requiring content snapshots, the platform maintains a complementary forensics_log table with a shorter retention window and stronger access controls.

Schema

```
CREATE TABLE public.forensics_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID REFERENCES public.tenants(id),  
  audit_log_id UUID REFERENCES public.audit_log(id),  
  snapshot_type TEXT NOT NULL CHECK (snapshot_type IN (  
    'commission_dispute',  
    'booking_dispute',
```

```

        'tenant_complaint',
        'ai_misbehavior_investigation',
        'data_export_request',
        'security_incident'
    )),
    reason TEXT NOT NULL,
    -- Application-layer encrypted with
FORENSICS_ENCRYPTION_KEY_CURRENT
    encrypted_payload BYTEA NOT NULL,
    encryption_key_id TEXT NOT NULL,
    redacted_summary JSONB,
    captured_by_user_id UUID REFERENCES public.users(id),
    captured_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    purge_after TIMESTAMPTZ NOT NULL,
    legal_hold BOOLEAN NOT NULL DEFAULT FALSE,
    last_accessed_at TIMESTAMPTZ,
    access_count INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX forensics_log_purge_idx ON
public.forensics_log(purge_after)
    WHERE legal_hold = FALSE;
CREATE INDEX forensics_log_audit_link_idx ON
public.forensics_log(audit_log_id);

```

Encryption

The forensics log uses application-layer encryption with a dedicated key (FORENSICS_ENCRYPTION_KEY_CURRENT per §28.13), separate from the §13.5 tenant-credentials key. Separating keys means:

- A compromise of the credentials key does not expose forensics content.
- A compromise of the forensics key does not expose tenant credentials.
- Each key has independent rotation cadence (forensics rotates every 180 days per §28.20).

The encryption uses AES-256-GCM with per-record IV. The same DR discipline from §13.5.3 applies — offsite backup, quarterly verification.

What gets captured

snapshot_type drives the capture shape. Examples:

- **commission_dispute**: snapshot of the disputed commission row, the underlying booking row, the host adapter response that determined the commission, the audit_log entries for any override actions, and the conversation messages where the customer was told about the commission rate (if applicable).
- **ai_misbehavior_investigation**: snapshot of the message, its preceding context (previous N messages), the persona prompt at the time, the retrieved RAG chunks, the supervisor findings, the customer's thumbs-down feedback if any.
- **data_export_request**: snapshot of every personal-data column for the customer at the moment of export. The export sent to the customer is reconstructable from this snapshot.

The capture functions live in `src/lib/forensics/capture.ts` and are invoked from event handlers — never automatically on every event (that’s the audit log’s role).

Access controls

Forensics log entries are accessible only to:

- Platform admin via `withPlatformAdminAudit` per §5.4.8 with `reason = 'forensics_log_review'`. Every access is itself audited.
- Authorized legal counsel via documented access procedure (NOT programmatic — manual decryption with operator-controlled keys, paired with a court order or signed engagement letter).

The forensics log is NEVER exposed through tenant-facing APIs. A tenant who wants their own forensic data uses the CCPA export path (§17.9).

Retention

- Default: 90 days from `captured_at`. After 90 days, `forensics_log_purge_cron` (nightly) deletes rows where `purge_after < NOW() AND legal_hold = FALSE`.
- Legal hold: setting `legal_hold = TRUE` (platform admin action) prevents purge until the hold is removed.
- Audit log retention (7 years per §26.5) is unaffected — the audit row pointing at the deleted forensics entry simply has a null `forensics_log_id` reference after purge.

Cost considerations

The forensics log is bounded by capture invocation. It is NOT a “log everything” mode — that would be storage-prohibitive and privacy-hostile. Captures happen:

- On dispute creation (automatic from `commissions.dispute_*` fields)
- On tenant complaint submission
- On manual platform-admin trigger (“snapshot this conversation for investigation”)
- On `security_incident` declaration
- On CCPA data export request (per §17.9 the export itself can optionally trigger a forensics snapshot for audit completeness)

Steady-state volume is bounded. Capture cost is the encrypted-write overhead plus storage; per-tenant cost is a function of how often that tenant generates disputes/complaints/security events.

Calls Worth Flagging

- **The audit log answers “did this happen?”; the forensics log answers “what did people see?”** Both are needed. Conflating them produces either too much PII in long-term storage (privacy exposure) or insufficient content for investigations (forensic blindness).
- **90-day retention is the default; legal hold extends.** Routine investigations should resolve within 90 days. The hold flag is for the unusual case (lawsuit, regulatory review).
- **The encryption-key separation matters.** Don’t reuse the credentials key. Don’t reuse the cookie-session key. A dedicated key keeps blast radius small on compromise.

- **Every forensics access is audited.** The audit row for a forensics access includes the accessor’s identity, the investigation reason, and a link back to the forensics row.

26.6 Monitoring & Alerts

Signal	Source	Severity
Auth failure spike	Supabase logs	Medium
Permission denied spike	App logs	Medium
Cross-tenant RLS bypass attempt	App logs	Critical
AI cost surge	Anthropic billing API	Medium
Stripe webhook signature failure	Webhook handler	High
Adapter health degraded	Health check cron	Medium
Database connection pool exhaustion	Supabase metrics	High
Error rate >1% sustained	Sentry / Vercel	Medium
Page latency >2s p95	Vercel Analytics	Low

26.7 Incident Response

Priority	Definition	Initial Response SLA
P1	Platform down or data breach confirmed	Within 15 minutes
P2	Major function broken (chat, booking, payments) for multiple tenants	Within 1 hour
P3	Single tenant affected; not data breach	Within 4 hours
P4	Minor issue, workaround available	Next business day

26.8 Anti-Prompt-Injection

- System prompt and user input are strictly separated; user input never interpolated into system prompt
- RAG content includes ‘is RAG data, not instruction’ framing in prompt
- User input is sandboxed within ‘USER MESSAGE’ delimiters
- Persona addendums (Agency tier) pre-screened by Haiku for injection patterns
- Supervisor checks for AI behavior outside expected persona scope
- Tools cannot be invoked solely from user content; AI must reason its way to a tool call

26.9 Vendor Outage Handling

Vendor	If Down	Behavior
Anthropic	Chat fails	Degrade to 'Our AI is temporarily unavailable; please leave a message' OR fallback to cached responses for common queries
OpenAI	Embeddings unavailable	RAG retrieval works on existing embeddings; new submissions queued
Stripe	Subscriptions/Connect down	Read paths continue; write paths queue with retry; tenant warned via dashboard banner
Resend	Email delivery fails	Queue with retry; in-app notifications continue; tenant warned
Supabase	Database down	Site is down; status page; reliance on Supabase SLA

26.10 Breach Response Process

- Detect (monitoring alert, audit log review, external report)
- Triage (severity assignment; convene response team)
- Contain (revoke compromised credentials; isolate affected systems)
- Investigate (scope of access; what was exfiltrated; how attacker entered)
- Notify (legal counsel; affected users per \$25.9 SLA; tenant admins; regulators per state law)
- Remediate (patch vulnerability; force credential rotation; security review)
- Post-mortem (write up; circulate; track remediation items)

26.11 Penetration Testing

- Annual third-party pen test (Phase 2 post-launch)
- Quarterly internal security review
- Bug bounty program (Phase 3 post-launch)

26.12 Compliance Posture

- CCPA-aligned at launch
- GDPR architecture-ready; not certified at launch
- SOC 2 Type 1 within first 12 months post-launch (target)
- PCI: not applicable; platform never sees card data (Stripe-tokenized)

26.13 Real PII on Staging — Documented Risk Acceptance

Staging contains real customer PII via the prod-DB-copy mechanism described in §30.9.1 and the CI/CD Pipeline doc §5. This is a documented risk acceptance; the alternative (synthetic-only staging) was rejected because it would not catch a class of data-shape and data-volume bugs that the platform has explicitly chosen to test for.

Controls:

- **Outbound isolation.** Wrapper functions `src/lib/email/send.ts` and `src/lib/twilio/client.ts:sendSms()` apply `TEST_OVERRIDE_EMAIL` and `TEST_OVERRIDE_PHONE` to every outbound send. Audit of call sites is part of the one-time setup checklist (CI/CD doc §11). New outbound integrations must route through these wrappers as a code-review requirement.
- **External-service neutering.** OAuth tokens and Stripe customer/subscription IDs cleared by the post-restore fixup script (CI/CD doc §5.1). Staging cannot impersonate users or interact with live customers in external systems.
- **Background job suppression.** Inngest crons disabled on staging; the `email_messages` status field is set to ignored for unprocessed records on restore.
- **Access scope.** Staging access is restricted to the same users who have production access. No broader staging-only access exists.
- **Refresh hygiene.** Staging data is overwritten on each release. Customer deletion requests propagate to staging on the next release cycle.

Audit log scope (§26.5) and monitoring (§26.6) apply to staging as well as production. Cross-tenant RLS bypass attempts detected on staging are P1, the same as on production.

Section 27 — SaaS Abuse Monitoring & Cost Controls

The platform's cost structure is unusual: a single tenant's behavior (especially AI usage) can run up real dollars in third-party API spend within hours. This section defines how the platform monitors per-tenant usage across five dimensions, enforces a three-tier limit structure with dimension-specific thresholds, and surfaces the right signal to the right person at the right time.

27.1 Why This Section Exists

Three forces make abuse monitoring more than a ‘nice to have’:

- **Cost asymmetry.** ** **Anthropic and OpenAI charge per token. A single tenant looping a customer through novel-length conversations, or running a runaway script against the chat API, can spend hundreds of dollars in an afternoon. Subscription revenue from a single-seat tenant for the same period is at most \$249/month (Sub-Host Agency); multi-seat tenants on the seat ladder can exceed this, but the cost-to-revenue ratio is what matters and it does not improve at scale without limits. Without limits, a few bad actors can wipe out platform margin.
- **Sender reputation.** ** **Email blast abuse from any tenant on platform Resend infrastructure degrades deliverability for every tenant on the same sending domain. One tenant’s misbehavior is everyone’s problem.
- **Quality drift via RAG.** ** **A tenant flooding their knowledge base with low-quality or copy-paste content degrades retrieval quality for that tenant and (if promoted to global) for the platform.

The goal is not to police tenants — it is to catch the small subset whose usage patterns are inconsistent with the platform’s economics or with shared infrastructure health, and to do so before damage is done.

27.2 Five Monitored Dimensions

Dimension	Concern	Cost Type	Reset
AI cost (LLM tokens)	Tenant burns disproportionate AI spend	Direct \$	Monthly
RAG submissions	Bulk dump of low-quality tenant-specific content	Indirect (quality + storage)	Total cap with promotion-earned increases (no reset)
Chat message volume	Runaway scripts; customer-facing chat loops	Direct \$ (drives AI cost)	Monthly
Email send volume	Spam blasts; sender reputation degradation	Reputation + \$	Daily + monthly
Group invitation blasts	Coordinator inviting hundreds at once	Reputation + spam classification	Monthly

27.3 Three-Tier Limit Structure (Hybrid)

The structure is uniform across all five dimensions; the thresholds are dimension-specific.

Soft Limit 1 (most-used threshold):

Action: Throttle the offending activity (rate-limit or queue).

Platform admin notified (no customer-visible signal).

Tenant operations continue but slowed.

Soft Limit 2 (warning threshold):

Action: Tenant owner emailed with explanation and projection.

Throttle continues; harder if usage keeps climbing.

Platform admin sees escalation in dashboard.

Hard Limit (cutoff threshold):

Action: The offending dimension is paused for the rest of the current billing month.

Tenant owner notified by email + in-app banner.

Customer-facing impact varies by dimension (see 27.6).

Tenant must contact support to discuss raised limits or address root cause before the next month rolls over.

The same three-tier structure is used everywhere; only the numeric thresholds differ by dimension and tier. This keeps the runbook short and consistent for platform admin.

27.4 Thresholds by Dimension

All thresholds are per tenant per calendar month unless otherwise noted. Calendar month aligns with subscription billing periods, so monthly resets are predictable. Every tenant is evaluated independently — there's no concept of one tenant inheriting or sharing usage with another.

27.4.1 AI Cost

The most operationally important dimension. Thresholds are in platform-attributed AI cost — the dollar value of Anthropic + OpenAI usage attributed to that tenant.

Sizing rule: hard limit must sit below subscription revenue. This ensures the platform does not lose money on any tenant, even at peak abuse-pattern usage. The structure is calculated as a percentage of the tenant's effective monthly revenue:

- Soft 1 = 30% of effective monthly revenue
- Soft 2 = 50% of effective monthly revenue
- Hard = 70% of effective monthly revenue

The remaining 30% of revenue covers Stripe fees (~3%), non-AI infrastructure (Supabase, Vercel, Resend), platform operational overhead, and platform margin.

Effective monthly revenue is calculated from the tenant’s current subscription state:

- **Monthly billing:** effective monthly revenue = base seat price + sum of additional seat prices per \$3.3 ladder
- **Annual billing:** effective monthly revenue = (annual price ÷ 12). Because annual pricing is 10× monthly (two months free), the annual customer’s effective monthly revenue is 10/12 ≈ 83.3% of the equivalent monthly customer’s. Thresholds scale down accordingly.

Thresholds are recalculated whenever the tenant’s subscription state changes — seat added or removed, billing period switched, tier upgraded or downgraded.

Illustrative thresholds at single-seat base price, monthly billing:

Tier	Monthly Revenue	Soft 1 (30%)	Soft 2 (50%)	Hard (70%)
BYO Research	\$19	\$5.70	\$9.50	\$13.30
BYO Professional	\$59	\$17.70	\$29.50	\$41.30
BYO Agency	\$99	\$29.70	\$49.50	\$69.30
Sub-Host Starter	\$49	\$14.70	\$24.50	\$34.30
Sub-Host Pro	\$149	\$44.70	\$74.50	\$104.30
Sub-Host Agency	\$249	\$74.70	\$124.50	\$174.30

Worked example (multi-seat, annual): A Sub-Host Agency tenant on annual billing with 6 users has annual revenue of \$2,490 (base) + 3×\$590 (users 2-4) + 2×\$490 (users 5-6) = \$5,240/year. Effective monthly revenue = \$5,240 ÷ 12 = \$436.67. Thresholds: Soft 1 \$131.00, Soft 2 \$218.33, Hard \$305.67.

These percentages should be treated as starting estimates, not contractually fixed. Real-world usage data in the first 90 days post-launch should drive recalibration. The platform should also watch the ratio of cost-to-revenue across all active tenants — if median tenant cost runs at 80%+ of revenue, the subscription pricing itself needs adjustment, not just thresholds.

I’m not certain how Anthropic prices Sonnet vs. Haiku will hold through 2026; if pricing moves, these thresholds need to move with it.

27.4.2 RAG Submissions (Total Cap with Promotion Rewards)

This dimension is structurally different from the other four. The platform wants tenants to upload knowledge content — high-quality content makes the system better, and content useful enough to promote to global makes the platform better for everyone. So the model is designed to reward beneficial behavior, not just penalize volume.

Core mechanics

- The cap is a total cap on tenant-specific chunks, not a monthly velocity limit. It's cumulative across the tenant's lifetime.
- 'Tenant-specific' means chunks with AI-assessed relevance to global scope below 0.6 — content useful only to that tenant.
- Chunks promoted to global scope do not count against this cap (they belong to the platform now).
- Chunks sitting in the ingestion queue awaiting admin global-review decision (relevance ≥ 0.6) do not count while pending. They can sit there indefinitely.
- Each globally-promoted chunk permanently raises the tenant's cap by 25 chunks — a meaningful reward that compounds over time for tenants who contribute useful content.

Base caps by tier

Tier	Base Tenant-Specific Cap
BYO Research	200
BYO Professional	400
BYO Agency	800
Sub-Host Starter	200
Sub-Host Pro	600
Sub-Host Agency	1,500

For multi-seat tenants (BYO Agency, Sub-Host Agency), the base cap scales with effective monthly revenue using the same multiplier as §27.4.1 — adding seats raises the cap proportionally. Annual-billing tenants get caps at 10/12 $\approx 83.3\%$ of the equivalent monthly tenant's, consistent with their lower effective monthly revenue.

Effective cap = (base $\times$ revenue multiplier) + (25 $\times$ number of globally-promoted chunks from this tenant). Once raised by global promotion, the bonus persists even if the platform later removes the promoted chunk. This is a deliberate trust signal: 'you proved you contribute useful content, so we trust you with more capacity.'

When a submission lands

- Pipeline assigns relevance score (per §22.4 stage 4)
- If relevance $\geq 0.6 \rightarrow$ queued for admin global review, does not count against cap while pending
- If relevance < 0.6 AND tenant is under cap $\rightarrow$ approved as tenant-specific chunk, counts against cap
- If relevance < 0.6 AND tenant is at or over cap $\rightarrow$ chunk auto-deleted; user sees inline message in submission UI

When admin reviews a queue-eligible item

- Promote to global $\rightarrow$ chunk's scope becomes global; tenant's cap raises by 25; original counts are unchanged

- Decline → chunk falls back to tenant-specific scope. If tenant is under cap, chunk approved as tenant-specific and now counts. If tenant is at or over cap, chunk auto-deleted; tenant notified.

Warning structure (modified from the standard three-tier)

The standard ‘soft1 throttle → soft2 throttle → hard cutoff’ structure doesn’t fit here because the cap is enforced by auto-deletion rather than throttling. Instead:

State	Trigger	Behavior
Approaching	85% of effective cap	Banner to tenant: ‘You’re nearing your tenant-content cap. Submitting content that the platform can promote globally will increase it.’
At cap	100% of effective cap	Banner + email to tenant. New low-relevance submissions auto-deleted with inline notice. Platform admin notified for visibility (not action).
Over cap	Cap was higher previously, e.g. due to chunk removal	Same as at-cap behavior. No additional penalty.

There is no ‘hard cutoff’ tier here because over-cap behavior IS the enforcement — submissions don’t stop, they’re simply filtered. The tenant can keep submitting queue-eligible content; only tenant-specific landings get pruned.

Tenant UI cues

- Submission form shows current count vs effective cap, with breakdown of base cap + promotion bonus
- ‘You earned X additional slots by contributing Y chunks to the platform’ — concrete framing
- Approaching-cap banner explicitly suggests: ‘Submit content that’s useful beyond your agency — the platform admin reviews submissions for promotion to global scope, and each promotion earns you +25 slots permanently.’

This is the only dimension where the platform’s interest and the tenant’s interest are aligned by design: more genuinely useful content benefits both parties, and the cap is sized so legitimate tenant operation never hits it.

27.4.3 Chat Message Volume

Chat volume is the leading indicator for AI cost. The two correlate strongly but not perfectly (Haiku-heavy turns are cheaper than Sonnet-heavy ones). Both are monitored independently because

volume can spike without cost spiking (e.g., short turns dominated by classification) and vice versa.

Thresholds are sized so a tenant burning average cost-per-message would hit the chat volume threshold around the same time as the AI cost threshold. This gives the platform two independent triggers; whichever fires first wins. Because the AI cost threshold scales with effective monthly revenue (per §27.4.1), chat volume thresholds scale with the same multiplier.

Sizing rule: Chat volume thresholds = base allocation × (tenant effective monthly revenue ÷ tier reference revenue). The reference revenue is the single-seat monthly price for that tier. So a tenant adding seats sees thresholds rise proportionally; an annual customer sees thresholds at 10/12 ≈ 83.3% of the equivalent monthly customer.

Illustrative thresholds at single-seat base price, monthly billing:

Tier	Soft 1 (msgs/mo)	Soft 2 (msgs/mo)	Hard (msgs/mo)
BYO Research	600	1,000	1,500
BYO Professional	2,000	3,500	5,000
BYO Agency	3,500	6,000	8,500
Sub-Host Starter	1,500	2,800	4,300
Sub-Host Pro	5,000	8,800	12,500
Sub-Host Agency	8,500	14,300	21,400

Worked example (multi-seat, annual): Same Sub-Host Agency tenant from §27.4.1 — 6 users, annual billing, effective monthly revenue \$436.67. Reference revenue for Sub-Host Agency is \$249/month. Multiplier = $\$436.67 \div \$249 = 1.753$. Thresholds: Soft 1 ≈ 14,900 msgs/mo, Soft 2 ≈ 25,070 msgs/mo, Hard ≈ 37,520 msgs/mo.

‘Messages’ counts both user and assistant turns. Forum chat counts separately (no AI cost there) and is governed by per-user strike system in §19, not this dimension.

If AI pricing changes meaningfully (Anthropic or OpenAI), volume thresholds should be recalibrated alongside cost thresholds — the message-to-dollar ratio shifts.

27.4.4 Email Send Volume

Reputation is shared across tenants using platform Resend (Pattern B per §16.4). Tenants on Pattern A (their own Resend) are still monitored but the hard limit is advisory — their reputation, their problem — unless their sending domain CNAMEs back to a platform-shared address.

Sizing rule: Email volume thresholds = base allocation × (tenant effective monthly revenue ÷ tier reference revenue), same scaling rule as §27.4.1 and §27.4.3. A tenant adding seats earns proportionally more send headroom; an annual customer’s caps scale down to 10/12 ≈ 83.3% of the equivalent monthly customer’s.

Illustrative thresholds at single-seat base price, monthly billing:

Tier	Soft 1 (per day)	Soft 2 (per day)	Hard (per day)
BYO Research	100	200	400
BYO Professional	500	1,000	2,000
BYO Agency	1,000	2,000	4,000
Sub-Host Starter	150	300	600
Sub-Host Pro	1,250	2,500	5,000
Sub-Host Agency	2,850	5,700	10,700

Transactional emails (booking confirmations, password resets) are excluded from this count. The dimension is about marketing + notification email volume.

A separate signal — bounce rate above 5% in a rolling 24-hour window — escalates straight to platform admin regardless of volume, and may pause sending even before volume thresholds are hit. Hard bounces signal list quality issues that compound deliverability damage.

Note on shared-reputation risk: Because email volume thresholds scale with revenue, a large multi-seat tenant could in principle generate high send volume that affects shared-IP deliverability for smaller tenants. Operationally, the bounce-rate signal above is the second guardrail; if a high-volume tenant degrades the shared pool, the bounce check escalates them out of Pattern B faster than monthly volume alone would.

27.4.5 Group Invitation Blasts

A single group can invite a large number of people legitimately (a family reunion cruise might be 80 people). The abuse pattern is different: many groups in rapid succession from one tenant, or one group with an absurd invitee list (1,000+).

Tier	Soft 1	Soft 2	Hard
Any tier (BYO Pro and above; lower tiers have no group feature)	500	1,000	2,000

Tier doesn't matter much here — the abuse pattern is the same regardless of plan. A separate per-group cap of 100 invitees applies on top of the monthly tenant cap.

27.5 Schema

- Per-tenant per-month usage tracking (for monthly-reset dimensions)

```
CREATE TABLE public.tenant_usage_metrics (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON
DELETE RESTRICT,

billing_period DATERANGE NOT NULL,

ai_cost_cents BIGINT NOT NULL DEFAULT 0,

chat_messages_count INTEGER NOT NULL DEFAULT 0,

email_sent_count INTEGER NOT NULL DEFAULT 0,

group_invitees_count INTEGER NOT NULL DEFAULT 0,

email_sent_today INTEGER NOT NULL DEFAULT 0,

email_sent_day_ref DATE NOT NULL DEFAULT CURRENT_DATE,

ai_cost_limit_state TEXT NOT NULL DEFAULT 'ok'

        CHECK (ai_cost_limit_state IN
('ok', 'soft1', 'soft2', 'hard')),

ai_cost_state_changed_at TIMESTAMPTZ,

chat_volume_limit_state TEXT NOT NULL DEFAULT 'ok',

chat_volume_state_changed_at TIMESTAMPTZ,

email_volume_limit_state TEXT NOT NULL DEFAULT 'ok',

email_volume_state_changed_at TIMESTAMPTZ,

group_invite_limit_state TEXT NOT NULL DEFAULT 'ok',

group_invite_state_changed_at TIMESTAMPTZ,

last_recomputed_at TIMESTAMPTZ DEFAULT NOW(),

UNIQUE (tenant_id, billing_period)

);

```

- RAG content quota (separate table; total cap with promotion rewards, NOT monthly)

```

CREATE TABLE public.tenant_rag_quotas (

tenant_id UUID PRIMARY KEY REFERENCES public.tenants(id) ON
DELETE RESTRICT,

base_cap INTEGER NOT NULL,

promoted_chunks_count INTEGER NOT NULL DEFAULT 0,

current_tenant_chunks_count INTEGER NOT NULL DEFAULT 0,

rag_state TEXT NOT NULL DEFAULT 'ok'

        CHECK (rag_state IN
('ok', 'approaching', 'at_cap', 'over_cap')),

rag_state_changed_at TIMESTAMPTZ,

updated_at TIMESTAMPTZ DEFAULT NOW()

```

);

- Admin-granted exception

```
CREATE TABLE public.tenant_usage_overrides (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON  
  DELETE RESTRICT,  
  dimension TEXT NOT NULL CHECK (dimension IN (  
    'ai_cost', 'rag_cap', 'chat_volume', 'email_volume', 'group_invite'  
  )),  
  tier_override TEXT CHECK (tier_override IN  
    ('soft1', 'soft2', 'hard', 'base_cap')),  
  threshold_value BIGINT NOT NULL,  
  effective_from DATE NOT NULL,  
  effective_to DATE,  
  reason TEXT NOT NULL,  
  approved_by_user_id UUID NOT NULL REFERENCES  
  public.users(id),  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

- Audit trail for every state transition

```
CREATE TABLE public.usage_limit_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),  
  dimension TEXT NOT NULL,  
  from_state TEXT NOT NULL,  
  to_state TEXT NOT NULL,  
  metric_value BIGINT NOT NULL,  
  threshold_crossed BIGINT NOT NULL,  
  triggered_at TIMESTAMPTZ DEFAULT NOW(),  
  notification_sent_to TEXT[],  
  resolution_action TEXT,  
  resolved_at TIMESTAMPTZ,  
  resolved_by_user_id UUID REFERENCES public.users(id)  
);
```

- RAG-specific audit (auto-deletes, promotion bonuses, etc.)

```

CREATE TABLE public.tenant_rag_cap_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES public.tenants(id),
event_type TEXT NOT NULL CHECK (event_type IN (
'submission_auto_deleted','declined_chunk_auto_deleted',
'promoted_cap_increased','state_transition','manual_cap_adjustment'
)),
chunk_id UUID,
queue_id UUID,
cap_before INTEGER,
cap_after INTEGER,
count_before INTEGER,
count_after INTEGER,
reason TEXT,
occurred_at TIMESTAMPTZ DEFAULT NOW()
);

```

27.6 Enforcement Behavior by Dimension

What 'soft1 throttle / soft2 throttle / hard cutoff' actually means in user-visible terms varies by dimension:

Dimension	Soft 1 Throttle	Soft 2 Throttle	Hard Cutoff Effect
AI cost	Switch eligible turns from Sonnet to Haiku; cap conversation memory size injected into prompts	Above + reduce RAG top-k from default 4 to 2; cap response length	Chat falls back to 'Our system is over capacity for the moment — a human will be in touch.' All AI responses paused for tenant. Existing conversations remain readable.
RAG submissions	(Approaching state) Banner to tenant suggesting they submit promotion-worthy content to earn more capacity	(At cap state) New tenant-specific submissions auto-deleted with inline notice; queue-eligible submissions continue to	No traditional hard cutoff — over-cap enforcement is auto-deletion of low-relevance new content.

		flow normally	
Chat volume	Insert 2-3 second delay between user message and AI response	Insert longer delay; serve 'we're experiencing high load' message variant	Same as AI cost hard cutoff
Email volume	Queue email send with 5-minute delay	Queue with 30-minute delay	Send refused; transactional emails still go through; tenant sees banner
Group invitation	Delay invitation sends; require explicit 'Send Now' confirmation per batch	Same plus require platform admin pre-approval for batches > 20	No new invitations until next month; existing groups continue to function

The 'Hard cutoff' column is deliberately conservative — paused, not deleted. Existing conversations stay readable. Existing groups stay open. The tenant can keep using the parts of the platform that aren't in cutoff. The aim is to stop the bleeding, not to brick the tenant.

The RAG dimension is the only one without a hard-cutoff tier. By design: the platform's interest is more useful content, not less. Over-cap behavior filters out content that wouldn't have helped the platform anyway, while letting promotion-worthy content keep flowing.

27.7 Recomputation & State Transitions

Real-time (event-driven)

- Every AI API call appends to running `ai_cost_cents`
- Every chat message increments `chat_messages_count`
- Every email send increments `email_sent_count` and `email_sent_today`
- Every group invitation send increments `group_invitees_count`
- Every approved tenant-specific RAG chunk creation increments `current_tenant_chunks_count`
- Every promotion to global increments `promoted_chunks_count`
- Every chunk deletion decrements `current_tenant_chunks_count`

Scheduled (daily at 03:00 UTC)

- Reset `email_sent_today` to 0 if `email_sent_day_ref` is from a prior day
- Reset monthly metrics if `billing_period` has rolled over (new row inserted; old row preserved for audit). RAG quota is NOT reset — it persists across months.

- Recompute monthly state for tenants near thresholds

State transitions for monthly dimensions (AI cost, chat volume, email volume, group invitations) are monotonic within a billing period: a tenant can move ok → soft1 → soft2 → hard, but not backward until the next monthly reset. This prevents oscillation if usage drops just below a threshold and then climbs back up. To move backward early, a platform admin must explicitly resolve the usage_limit_event with notes.

State transitions for the RAG dimension are NOT monotonic — they reflect current standing. A tenant who deletes chunks to free up cap space immediately drops back to 'ok' status.

27.8 Notification Flow

Threshold crossed



soft1: soft2:

In-app banner to Email to tenant owner with:

platform admin only - which dimension

(no tenant notification — - current usage / threshold

throttle is invisible) - projected month-end usage

- actions tenant can take

Platform admin banner promoted

to "escalation watch" list

|

▼

hard:

Email + in-app banner to tenant owner

Email + Slack to platform admin

Tenant admin dashboard shows

red banner with link to support

27.9 Override Workflow

Platform admin may grant overrides via /admin/tenants/{id}/usage-overrides. Use cases:

- A tenant has a legitimate spike (large family-reunion group invitation, big seasonal campaign)
- A tenant is on a hand-negotiated comp arrangement
- A new feature rollout legitimately pushes a tenant past a threshold

Overrides are dimension-specific, time-bounded, and require a written reason logged in the override row. They do not silently raise the threshold forever — there's an `effective_to` date and an audit trail.

27.10 Platform Admin Dashboard

A dedicated view at `/admin/abuse-monitoring` shows:

- All tenants currently at soft1, soft2, or hard on any dimension
- Trend lines (per-tenant usage curves overlaid on threshold bands)
- Tenants approaching thresholds (e.g., >85% of soft1)
- Recent state transitions (last 30 days)
- Override requests pending review
- 'Top consumers' lists per dimension (useful for spotting anomalies before they breach)

The dashboard is read-only-by-default; making changes (overrides, manual state resets) requires `platform_super_admin` role and re-auth per §26.2.

27.11 Tenant-Facing Usage UI

Tenants see their own usage at `/settings/usage`:

- Current month's metrics with progress bars against their tier's thresholds
- Daily history for velocity dimensions
- Estimated month-end if current rate continues
- Plain-language explanation of what each limit means
- Link to upgrade their tier if they're consistently near thresholds

The aim is transparency: tenants should not be surprised when they hit a soft2 email, and should be able to plan their usage.

27.12 Cost Attribution

AI cost attribution requires care. A single chat turn typically involves:

- 1-2 Haiku calls (entity extraction, screening)
- 1 Sonnet call (main response)
- 1 supervisor Haiku call (preflight)
- Sometimes 1 OpenAI embeddings call (if RAG runs)

Each is logged with: `tenant_id`, `conversation_id`, `model`, `input_tokens`, `output_tokens`, `cost_estimate_cents`. The `cost_estimate_cents` is computed from the model's posted pricing at the time of the call (cached daily from Anthropic and OpenAI docs).

Memory extraction Haiku calls (per §11.2) attribute to the tenant whose memory is being extracted. RAG normalization Haiku calls attribute to the submitting tenant. Pre-cruise email generation attributes to the tenant whose customer is receiving the email.

The accuracy ceiling here is 'good enough for monthly billing reconciliation,' not 'good enough for chargeback to the cent.' Some shared overhead (model warm-up, background classification) is amortized.

27.13 Integration with Existing Sections

- §10 (Supervisor): supervisor preflight cost counts toward AI cost dimension
- §17 (Auth): signing in costs nothing; signups do nothing here
- §19 (Forum): forum strike system handles per-user abuse independently
- §22 (Knowledge Ingestion): velocity tracked here; quality checks happen in pipeline
- §23 (Email): suppressions are about deliverability per recipient; volume here is about tenant-level send pattern
- §24 (Chat UI): when chat is throttled or paused, banner copy comes from this section's enforcement table

27.14 What Is Not Covered Here

- Per-customer abuse (one customer spamming the chat): handled by per-IP and per-user rate limits in middleware
- Forum-level abuse: §19's strike system handles it
- Persona prompt addendum abuse: §16.6 Haiku pre-screen handles it
- Stripe payment fraud / chargeback abuse: handled by Stripe's risk engine
- Custom domain abuse (typosquatting, impersonation): domain verification per §16.3 catches most

27.15 Calls Worth Flagging

- **AI cost hard limits now sit below subscription revenue across all tiers.** ** **This guarantees the platform doesn't lose money on any tenant, even at peak abuse-pattern usage. The trade-off is less headroom for legitimate heavy users — if real-world data shows many tenants legitimately running near hard, the choice is to raise pricing or grant overrides, not to let cost exceed revenue.